ARL Protocol security review report

Version	1.0
Date	17 October 2026
Reviewed commit	`ab7595bc39c9256641fffc70103575c03a1cce9d`
Fixes	PR #55 (runbook procedure for M-03 and L-01)
Carried out by	The ARL team, with open-source tools and AI-assisted review. This is not a third-party audit.

Summary

All evidence (raw tool outputs, triage, findings) is public at github.com/armanalabsai/arlcoin.io/tree/main/security/2026-10-17.

This review was multi-layer and built on open-source tools. It covered the nine ARL contracts, the deployment scripts and the signed Base Mainnet run for the launch on 23 October 2026.

- **No critical finding, and no exploitable vulnerability in contract code.**
- **Contract design.** The token has no owner, mint, pause or upgrade. There is no proxy and no `delegatecall`.
- **Deployment run.** All four transactions in `apps/dapp/public/plans/8453-deploy.json` carry the bytecode built from the reviewed commit, and their constructor arguments equal `docs/mainnet-plan.md`.
- **Findings.** 1 high, 4 medium, 3 low and 4 informational, all concerning custody, launch procedure or documented design choices. Every finding is closed: fixed in the launch procedure, or mitigated or accepted with the reason recorded (`findings.md`).

Layers

Layer	Tool	Result	Evidence
Static analysis	Slither 0.11.6, 102 detectors	0 results	`static/slither.json`, `static/slither.sarif`
Static analysis	Aderyn 0.6.8	16 results (2 H, 14 L), all false positive or by design	`static/aderyn.json`, `static/aderyn.sarif`, `static/aderyn.md`
Unit, fuzz, invariant	Foundry 1.8.3	271/271 pass; 29 fuzz tests at 10,000 runs; 5 invariant suites, 32,768 calls each, 0 reverts	`dynamic/forge-test.json.gz`
Long campaign	Foundry, `deep` profile	271/271 pass; 200,000 runs per fuzz test; invariants 2,000 × 256 (512,000 calls each)	`dynamic/forge-test-deep.json.gz`
Symbolic proofs	Halmos 0.3.3	11/11 properties proved (bounded)	`symbolic/halmos.json`
Symbolic execution	Mythril 0.24.8 on runtime bytecode (TGE contracts)	No exploitable issue (see triage)	`symbolic/mythril-*.json.gz`
Fork rehearsal	Base Mainnet fork	Full launch day passes with the real signers	`docs/mainnet-runbook.md`

Layer	Tool	Result	Evidence
Manual and AI triage	ARL team	12 findings, 0 open	`triage.md`, `findings.md`

Areas checked

Area	Result
Access control and ownership	Token: none. Vesting: beneficiary fixed. Timelock: no external admin, 48-hour floor, independent canceller. Custody risk: H-01, M-02.
Supply and ERC-20 logic	Fixed 21,000,000 ARL minted once. Unmodified OpenZeppelin ERC20 and ERC20Permit. No-supply-change proved by Halmos.
Reentrancy and external calls	`nonReentrant` or checks-effects-interactions everywhere. ARL has no hooks.
Arithmetic	Solidity 0.8 checked arithmetic. Escrow amounts bounded to `uint128`.
Visibility	ABI of the token is pinned by CI (`scripts/check-token-abi.mjs`).
Events	Every state change emits an event.
Gas and DoS	No unbounded loops over user data in TGE contracts. Escrow push payments: L-02.
Base and bridges	ARL is native to Base. No bridge contract.
Upgradeability	None: no proxy, so no storage layout risk.

Findings

See `findings.md` for the table and the details of each finding, and `triage.md` for every tool result and why it was or was not a finding.

Reproduce

```
cd contracts
slither . --foundry-compile-all --filter-paths "lib/|test/" --exclude-dependencies --json slither.
json --sarif slither.sarif
aderyn . --src src -o aderyn.json
forge test --json > forge-test.json
FOUNDRY_DYNAMIC_TEST_LINKING=false halmos --match-contract ARLSymbolic --solver-timeout-assertion
120000
```

`.github/workflows/security-review.yml` runs the same checks on every change and on `main`:

- Slither must report nothing.
- Every Aderyn result must be in the triaged baseline (`security/aderyn-baseline.json`).
- Every test must pass, and every fuzz test must run at least 10,000 times.
- Halmos must prove every property.
- Mythril runs weekly.

Limits

- This is a review by the project team, not an independent audit.
- The Halmos proofs are bounded, not full formal verification.
- The internals of OpenZeppelin, Safe and Uniswap v3 are trusted as published.
- Off-chain components and market risk are out of scope (`scope.md`).

Appendix: Findings

Severity: Critical, High, Medium, Low, Informational.

Status values:

- **Fixed:** changed in the repository.
- **Mitigated:** the risk is reduced by a procedure or a configuration.
- **Accepted:** an owner decision, with the reason recorded.

No critical finding, and no vulnerability in contract code that an attacker could exploit.

ID	Severity	Title	Location	Status
H-01	High	About 79% of the supply is held by Safes that share the same three owners, 2-of-3	`contracts/script/ARLDeployer.sol:74-88`, `docs/mainnet-plan.md`	Mitigated (custody procedure)
M-01	Medium	Launch liquidity positions are held by the Safes, not locked on chain	`packages/deploy/src/pool.ts` (mint recipient)	Accepted (12-month hold policy)
M-02	Medium	A 1-of-1 guardian can cancel every treasury operation, including its own removal	`contracts/src/ARLTimelock.sol:43`	Mitigated (guardian key held offline)
M-03	Medium	The Public Launch batch reverts if anyone trades between the two pool batches	`packages/deploy/src/pool.ts:349`, runbook step 7	Fixed (runbook, PR #55)
M-04	Medium	Founder allocation unlocked at TGE; Team pool not yet in vesting wallets	`contracts/src/ARLAllocation.sol:20,23`	Accepted (owner decision, public)
L-01	Low	The deploy screen checks creation code but not constructor arguments	`apps/dapp/lib/deploySteps.ts:154-168`	Fixed (runbook check, PR #55)
L-02	Low	Push payments in escrows: a blocklisting token could block both parties	`contracts/src/ComputePayment.sol:265-266`, `ARLJobs.sol:181,238`	Accepted (ARL only; not deployed at TGE)
L-03	Low	Signal group roots are set by the group admin and not derived on chain	`contracts/src/ARLAnonymousSignal.sol:112-137`	Accepted (documented trust model)
I-01	Info	ERC-20 `approve` race; `permit` front-running	OpenZeppelin `ERC20`, `ERC20Permit`	Accepted (standard behavior)
I-02	Info	Tokens sent to a vesting wallet later vest on its schedule; lost beneficiary keys are unrecoverable	`contracts/src/ARLVestingWallet.sol`	Accepted (documented)
I-03	Info	After expiry, a client can refund a submitted but unevaluated job	`contracts/src/ARLJobs.sol:204-214`	Accepted (ERC-8183 model)
I-04	Info	Staking rounding dust and direct transfers stay in the contract	`contracts/src/ARLStakingRewards.sol`	Accepted (documented)

Details

H-01 Shared custody of the role Safes

- **What happens.** All eleven role Safes have the same three owners at a 2-of-3 threshold. At TGE they hold 16.5M ARL directly. The 1M ARL treasury sits behind the 48-hour timelock, and the 3.5M ARL in vesting release to Safes with the same owners.
- **Failure path.** Two owner keys are compromised, for example by phishing or a shared backup. The holder of those keys can move every Safe balance and the pool positions at once.
- **Mitigation.**
  - Each owner signs from a separate hardware wallet on a separate device, with an offline seed.
  - No two owners share a device or a backup.
  - After launch, the large Safes (Public Launch, Community & Staking, Ecosystem & Growth) move to 3-of-5 with independent signers.
  - This is a Safe configuration; no contract change is needed.

M-01 Liquidity positions not locked on chain

- **What happens.** The Uniswap v3 position NFTs are minted to the Liquidity and Public Launch Safes. The 12-month hold is a policy (`docs/launch-route.md`), so 2 of 3 owners can withdraw at any time.
- **Option kept for later.** Transferring the position NFTs to `ARLTimelock` would make every withdrawal visible 48 hours ahead and cancellable by the guardian. No new code is needed. The decision stays with the owners.

M-02 Guardian can freeze the treasury

- **What happens.** `CANCELLER_ROLE` can cancel any pending operation, including one that revokes the guardian.
- **Failure path.** A hostile or compromised guardian freezes 1M ARL. It cannot move those funds.
- **Mitigation.**
  - The guardian key is held offline by a person who is not a role-Safe owner.
  - The guardian Safe can later be raised to 2-of-3 without a contract change.

M-03 Public Launch batch ordering

- **What happens.** Each mint requires 99.9% of its ARL amount. Once someone trades, the price is inside the range and an ARL-only mint fails. Nothing is deposited.
- **Fix.** The runbook now requires collecting both owner signatures for the Public Launch batch before the Liquidity batch is executed, and executing it within seconds.

M-04 Founder and Team unlock

- **What happens.**
  - The Founder Safe receives 2.1M ARL unlocked at TGE.
  - The Team pool, 900k ARL, is assigned per grant into vesting wallets after TGE (`docs/tokenomics-economic-spec.md`).
- **Status.** Both are owner decisions and both are published.

L-01 Deploy arguments

- **What happens.** A contract address follows from the deployer and nonce alone, so a replaced run file with different constructor arguments would still produce the expected addresses.
- **Fix.** The runbook now requires two checks before signing: the run file's SHA-256, and each displayed argument against `docs/mainnet-plan.md`.

Appendix: Scope

Security review of 2026-10-17. Reviewed commit: `ab7595bc39c9256641fffc70103575c03a1cce9d` (`main`, after PR #54).

Contracts

File	SHA-256	On Base Mainnet at TGE
`contracts/src/ARLToken.sol`	`374a7cd0ffcca6eac87996d7a6328a2d5fce4bb16661b1eb06f02b15fa8893c8`	yes
`contracts/src/ARLAllocation.sol`	`49c25087b7e87a3d2693f2d54202b6bcbce41d468c84faaa61f696d0e768fdb2`	yes (library)
`contracts/src/ARLVestingWallet.sol`	`a5b291daf1366233eb10cc05d6fc624d1d1c7e8ac26919fa7e704e8978a93e2d`	yes (2 wallets)
`contracts/src/ARLTimelock.sol`	`7aed212888c032f553e9d33eb2fb7f122d790492eb1fc5bccb5f54617ed855f5`	yes
`contracts/src/ARLStakingRewards.sol`	`2bfcb880b0848abc039a61684be0cad870b56b45d4b10005fdbefe3a111027d3`	no (Base Sepolia)
`contracts/src/ARLMerkleDistributor.sol`	`7aa8d3959986ee840dc3536fea355970cbe6589638b887213d258c52c44a1762`	no (later airdrop)
`contracts/src/ComputePayment.sol`	`774a225e24fbaa6b6622ba7418cb62777a8274aed308372299e5694f9b066d24`	no
`contracts/src/ARLJobs.sol`	`a455e849e6d1b3a4ee05549da6033d622ffd879cf3b9d943b45ba7f6f21f22fd`	no
`contracts/src/ARLAnonymousSignal.sol`	`a093a361e07d882db1f7db596e750a0a79caf4fdeadce531cfe870a1c24d45bd`	no
`contracts/zk/ARLSemaphoreVerifier.sol`	`7336b1359bc1d49251a143373a96c97861c697bc1f9924459c97b3187ca64935`	no (generated)

Also reviewed: the deployment scripts (`contracts/script/DeployARL.s.sol`, `ARLDeployer.sol`, `ARLDeployPlan.sol`, `ARLVerify.sol`), the signed Base Mainnet run `apps/dapp/public/plans/8453-deploy.json` (SHA-256 `4e7acae253947d3343daeed8a56b893459d1b7797c2e7045bb2204ebfd6d909a`), the deploy screen's run parser (`apps/dapp/lib/deploySteps.ts`) and the Uniswap v3 batch builder (`packages/deploy/src/pool.ts`).

Build

solc 0.8.36, optimizer 200 runs, EVM cancun, no via-IR. OpenZeppelin Contracts v5.6.1 at `5fd1781b1454fd1ef8e722282f86f9293cacf256`, unmodified. Foundry 1.8.3.

Deployment run check

The creation code of all four transactions in `8453-deploy.json` equals the bytecode built from the reviewed commit, and their constructor arguments equal `docs/mainnet-plan.md`:

- **Vesting wallets (nonces 12, 13).** Beneficiaries: Investors Safe and Strategic Partnerships Safe. Cliff start 1792713600 (2026-10-23), cliff end 1824249600 (2027-10-23), vesting end 1918944000 (2030-10-23).
- **Timelock (nonce 14).** Delay 172800 s. Proposer and executor: the Treasury Safe only. Guardian: `0xB521d1E4eF0dA060e3B6C5d8667DE711abc4D120`.
- **Token (nonce 15).** The eleven recipients in canonical order, with the two vesting wallets and the timelock in their slots.

Out of scope

- Safe v1.5.0, OpenZeppelin and Uniswap v3 internals (used unmodified).
- The Noir circuit and the Barretenberg verifier generator.
- Off-chain components: website, dApp hosting, Telegram bot, x402 facilitator, DNS.
- Market and tokenomics analysis, and legal questions.
- Full formal verification. The Halmos checks are bounded.

Appendix: Triage notes

Every tool result was checked by hand against the code, with AI-assisted review. A result counts as a finding only if a realistic caller or input reaches it and it causes a loss or an operational failure. Everything else is a false positive or by design, and the reason is recorded here.

Slither 0.11.6 (`static/slither.json`, `static/slither.sarif`)

0 results with 102 detectors on 126 contracts (ARL code and deployment scripts; `lib/` and `test/` excluded). Twelve detectors are silenced inline with `slither-disable-next-line`, each with a reason next to it:

- **`timestamp`.** Reward periods, vesting and claim windows run for days. Drift of a few seconds does not change any outcome.
- **`incorrect-equality`.** These compare computed rates or amounts, never balances.
- **`reentrancy-benign` in `stakeWithPermit`.** The permit call goes to the immutable ARL token, which has no hooks, and the function is `nonReentrant`.

Aderyn 0.6.8 (`static/aderyn.json`, `static/aderyn.sarif`, `static/aderyn.md`)

Aderyn result	Verdict	Reason
H `contract-locks-ether`, `ARLVestingWallet.sol:22`	False positive	The inherited OpenZeppelin `VestingWallet.release()` sends ETH to the beneficiary. Aderyn does not follow the inherited function.
H `reentrancy-state-change`, `ARLAnonymousSignal.sol:175`	False positive	`verifier.verify` is declared `view`, so the compiler issues a STATICCALL that cannot change state. The verifier address is immutable.
L `large-numeric-literal`, `ARLAllocation.sol` (12 instances)	Accepted	Allocation constants written as whole-token amounts times `UNIT`. A test in `packages/tokenomics` checks them against the TypeScript table.
L `require-revert-in-loop`, `ARLAnonymousSignal.sol:201`	By design	One invalid commitment must revert the whole call.
L `unchecked-return`, `ARLTimelock.sol:43`	False positive	`_grantRole` returns false only if the role is already held. The constructor rejects a guardian that is also a proposer or executor, so the grant is always new.

Foundry fuzz and invariant (`dynamic/forge-test.json.gz`)

271 of 271 tests pass.

- **Fuzz.** 29 fuzz tests at 10,000 runs each (seed `0xa121`).
- **Invariant.** 5 invariant suites, each 256 runs × 128 depth = 32,768 calls, 0 reverts:
  - allocations reconcile;
  - each distributor entry pays exactly once;
  - the jobs balance equals the open escrow;
  - the staking accounting identity holds;
  - the compute balance equals the live escrow.

The long campaign (`FOUNDRY_PROFILE=deep`) also passed 271 of 271: 200,000 runs per fuzz test and 2,000 × 256 invariant runs (512,000 calls per suite). Result in `dynamic/forge-test-deep.json.gz`.

Halmos 0.3.3 (`symbolic/halmos.json`)

11 of 11 properties proved within bounds:

- no call changes the supply;
- `transfer` conserves supply and balances;
- `transferFrom` never exceeds the allowance;
- the vesting beneficiary is immutable;
- nothing vests before the cliff end;
- vesting is monotonic, never exceeds the allocation, and is complete at the end;
- the timelock delay floor holds;
- distributor claims pay only listed entries, once;
- sweeps go only to the return address, and only after the end.

Mythril 0.24.8 (`symbolic/mythril-*.json.gz`)

Mythril runs symbolic execution on the runtime bytecode of the three contracts deployed at TGE, with 2 transactions deep.

Contract	Mythril result	Verdict	Reason
`ARLVestingWallet`	SWC-107 external call to user-supplied address (`release(token)`)	False positive	OpenZeppelin updates `_erc20Released[token]` before the transfer. A caller-chosen token can only affect its own entry, and the payout always goes to the fixed beneficiary.
`ARLVestingWallet`	SWC-113 DoS with failed call	Accepted	ETH release reverts only if the beneficiary cannot receive ETH. The beneficiary is a Safe, which can, and ARL releases are unaffected.
`ARLVestingWallet`	SWC-116 timestamp dependence (×2)	By design	The vesting schedule is time-based by definition.
`ARLToken`	SWC-116 timestamp dependence in `permit (0xd505accf)`	By design	EIP-2612 requires the signature deadline to be compared with `block.timestamp`.
`ARLTimelock`	SWC-101 High, underflow in `onERC1155BatchReceived (0xbc197c81)`	False positive	This is the same result as in the earlier review (`docs/review-scope.md`): an artefact of the ABI decoder on malformed calldata. The call only returns the selector and changes no state. A test (`test_MythrilSwc101CalldataChangesNothing`) replays Mythril's transaction and a fuzz test sends arbitrary calldata; both pass.

Manual review

The manual review covered access control, supply, external calls, arithmetic, visibility, events, gas and DoS, bridge exposure (none: ARL is native to Base) and proxies (none). The results are in `findings.md`.

Two hypotheses were checked and rejected:

- **Nullifier aliasing (`n` and `n + p`) in `ARLAnonymousSignal`.** The verifier rejects public inputs at or above the field modulus (`zk/ARLSemaphoreVerifier.sol:577`).
- **Pool front-running at initialisation.** A pre-created pool above 0.20 USD makes the batch revert with nothing deposited (fork-tested). A pre-created pool below 0.20 USD is harmless, because the ARL-only range sits above the price.